National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

# RAI-833 Deep Learning

## Tutorial 1: Building a Perceptron

*Building from Scratch & Activation Function Modification*

*Submitted to:*

**Dr. Shahbaz Khan**

*Submitted by:*

**Hamza Suhaib Qarni**

**538866**

**SMME**

**February 24, 2026**

# 1 Objectives

The primary focus of this tutorial was to move beyond high-level libraries and understand the fundamental mechanics of a **single-layer neural network**. Key learning outcomes and tasks included:

- Implementing a **Perceptron** class from scratch using **Python** and **NumPy**.
- Understanding the **weight and bias update mechanics** (Gradient Descent / Delta Rule) through manual iteration.
- Replacing the standard Step Function with a **Sigmoid Activation Function** (Task 2).
- Modifying classification labels from $\{-1, 1\}$ to $\{0, 1\}$ (Task 3).
- Evaluating the model on the **Iris Dataset** and testing **manual feature inputs** (Task 1).

# 2 Methodology

The Perceptron was designed as an object-oriented class with two primary methods: `fit` (for training) and `predict` (for inference).

## 2.1 The Learning Logic

The implementation follows a standard gradient-based learning approach:

- **Initialization:** The model initializes all **weights to zero**.
- **Iteration:** During training, the algorithm sequentially iterates through the provided dataset.
- **Calculation & Update:** For every input, it calculates a **weighted sum ($z$)** and updates the weights based on the **prediction error**:

$$z = \sum_{j=1}^{m} w_j x_j + w_0 \tag{1}$$

$$W_{new} = W_{old} + \eta \cdot (\text{target} - \text{prediction}) \cdot x \tag{2}$$

Where $\eta$ represents the **learning rate**.

## 2.2 Activation Function Modification

To fulfill the specific tutorial tasks, the default **hard-limit step function** was replaced with the **Sigmoid function**. This allows the model to output a **continuous probability** between 0.0 and 1.0 rather than discrete integers.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

To verify the mathematical behavior of the activation function, the following Python snippet was executed to plot the Sigmoid curve and establish the 0.5 **classification threshold**:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the Sigmoid function
def sigmoid(z):
return 1.0 / (1.0 + np.exp(-z))

# Create a range of values for z
z = np.linspace(-7, 7, 200)
phi_z = sigmoid(z)

# Plot the Sigmoid curve and threshold
plt.plot(z, phi_z, color='blue', linewidth=2, label='Sigmoid $\sigma(z)$')
plt.axhline(y=0.5, color='red', linestyle='--', label='Threshold (0.5)')
plt.show()
```

Listing 1: Python implementation to define and visualize the Sigmoid mathematical function.

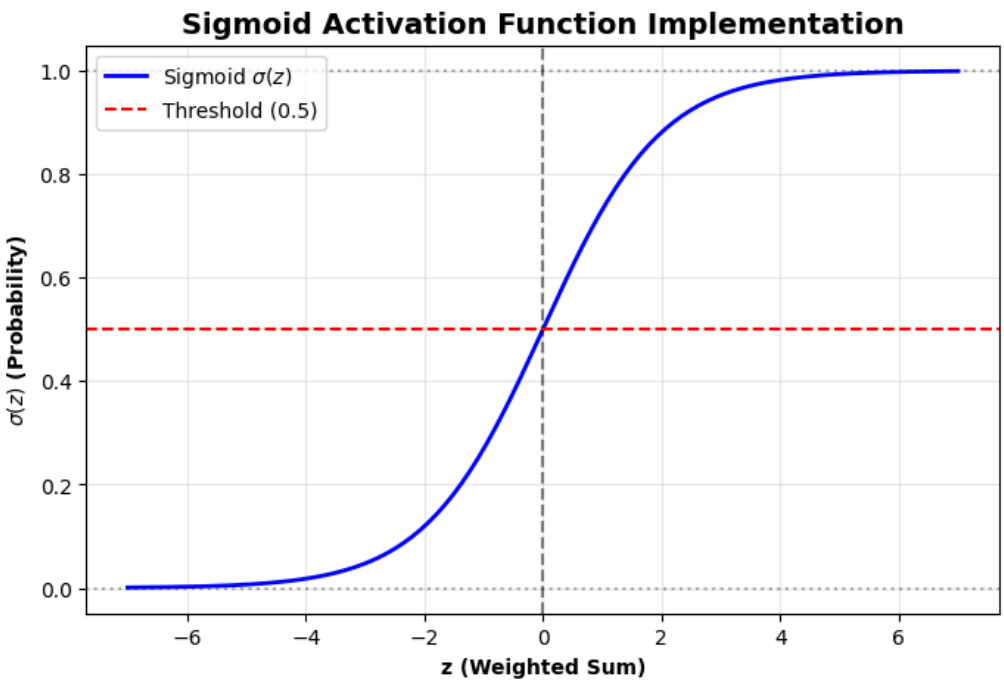


Figure 1: Visualization of the implemented **Sigmoid Activation function**. A threshold of **0.5** was established to convert the continuous probability into a binary classification result (0 or 1).

Table 1: Perceptron Configuration Comparison

| Feature | Original Perceptron | Modified Implementation |
|---|---|---|
| **Activation Function** | Step Function (Heaviside) | **Sigmoid Function** |
| **Output Type** | Discrete ($-1$ or $1$) | **Probabilistic** ($0.0$ **to** $1.0$) |
| **Class Labels** | $-1$ and $1$ | $0$ **and** $1$ |
| **Decision Boundary** | $z \geq 0.0$ | $\sigma(z) \geq 0.5$ |

# 3 Implementation Code

Below is the core Python implementation demonstrating the **manual weight updates** and the integration of the **Sigmoid activation function**.

```
class Perceptron(object):
def __init__(self, eta=0.01, n_iter=10):
self.eta = eta
self.n_iter = n_iter

def weighted_sum(self, X):
return np.dot(X, self.w_[1:]) + self.w_[0]

def predict(self, X):
# 1. Get the weighted sum
z = self.weighted_sum(X)

# 2. Apply the Sigmoid mathematical formula
sigmoid_output = 1.0 / (1.0 + np.exp(-z))

# 3. Return 1 if probability is >= 0.5, otherwise 0
return np.where(sigmoid_output >= 0.5, 1, 0)

def fit(self, X, y):
self.w_ = np.zeros(1 + X.shape[1])
self.errors_ = []
for _ in range(self.n_iter):
errors = 0
for xi, target in zip(X, y):
y_pred = self.predict(xi)
update = self.eta * (target - y_pred)
self.w_[1:] += update * xi
self.w_[0] += update
errors += int(update != 0.0)
self.errors_.append(errors)
return self

```

Listing 2: Custom Perceptron implementation fulfilling Tasks 2 and 3.

# 4 Results and Discussion

The modified model was trained on the **Iris Dataset**, specifically tasked with performing a binary classification to identify **Iris-setosa** (labeled as **1**) against all other species

(labeled as **0**).

### 4.1 Model Performance

The training yielded excellent convergence results:

- **Hyperparameters:** The model utilized a **learning rate ($\eta$)** of 0.1 and ran for 10 **iterations**.
- **Accuracy:** The algorithm converged efficiently, achieving a flawless testing accuracy of **100.0%**.
- **Justification:** Such high accuracy is attainable because the **Iris-setosa** class is **linearly separable** from the other species in the four-dimensional feature space. This mathematical separation allows the Perceptron to establish a perfect **decision boundary**.

### 4.2 Manual Inference (Task 1)

To test the practical utility of the model as required by **Task 1**, arbitrary manual feature values for *sepal length, sepal width, petal length, and petal width* were fed directly into the model:

- **Input Array:** `[5.0, 3.4, 1.5, 0.2]`
- **Model Prediction Output:** `[1]`
- **Decoded Result: Iris setosa**

The model successfully interpreted the manual array structure utilizing the `ast.literal_eval` module and correctly classified the sample.

## 5 Conclusion

This tutorial successfully demonstrated the foundational mechanics of deep learning. Key takeaways include:

- **Manual Implementation:** By building the Perceptron from scratch rather than relying on high-level APIs like Scikit-Learn, the step-by-step mathematical update of weights and biases was observed directly.
- **Bridging Concepts:** The successful replacement of the step function with the continuous **Sigmoid function** conceptually bridged the gap between a rudimentary linear classifier and modern logistic regression.
- **Performance Validation:** The architecture maintained **perfect classification accuracy (100%)** on linearly separable data, proving the validity of the custom weight-update loop.